1 Regular expressions

- An alternative way to describe regular languages
- Common in scripting languages (perl, python, etc.) & UNIX utilities (awk, sed, grep, etc.)
- Defined by the "regular operations":
  - Union ($\cup$ or $+$)
  - Concatenation ($\cdot$ or nothing)
  - Star ($*$).
    - Note: $+$ is similar to star; except that it cannot include empty set: $R^+ = R^* - \{\emptyset\}$.

Example

- 0^*10^* = $\{w | w\ contains\ exactly\ one\ 1\}$
- $\Sigma^*1\Sigma^*$ = $\{w | w\ contains\ at\ least\ one\ 1\}$
- $\Sigma^*001\Sigma^*$ = $\{w | w\ contains\ 001\ as\ a\ substring\}$
- $(\Sigma\Sigma)^*$ = $\{w | w\ contains\ an\ even\ number\ of\ characters\}$
- $\Sigma\Sigma^*$ = $\{w | w \neq \epsilon\}$
- $01 + 11$ = $\{01, 11\}$ = $(0 + 1)1$
- $\emptyset^*$ = $\{\epsilon\}$

Precedence of operators

1. Star
2. Concatenation
3. Union

Formal Definition:
R is a *regular expression* if R is one of the following:

- a, $a \in \Sigma$
- ϵ
- $\emptyset$
- $R_1 + R_2$
- $R_1 \cdot R_2$
- R_1^*

1.1 Equivalence of REs and FAs

Theorem 1.1 *A language is regular iff it can be generated by some Regular Expression.*

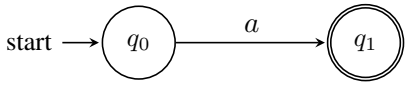
This will be proven in two parts:

Lemma 1.2 *Every language generated by a RE can be generated by some DFA/NFA.*

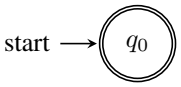
Lemma 1.3 *Every language generated by a DFA/NFA can be generated by some RE.*

Proof of Lemma 1.2. We will give an NFA for each of the 6 cases that define a RE.

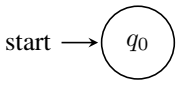
- $R = a$ for some $a \in \Sigma$



- $R = \epsilon$



- $R = \emptyset$

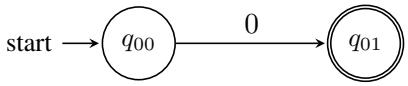


The NFA for the remainder can be constructed just in the same way as the NFA constructions we gave for the regular language operators:

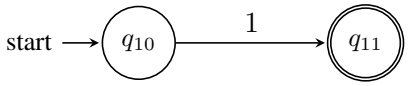
- $R_1 + R_2$
- $R_1 \cdot R_2$
- R_1^* ∎

Example

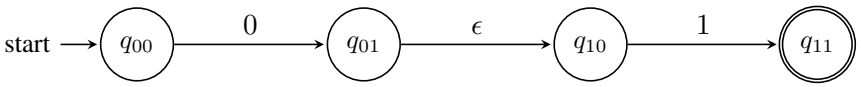
- 0:



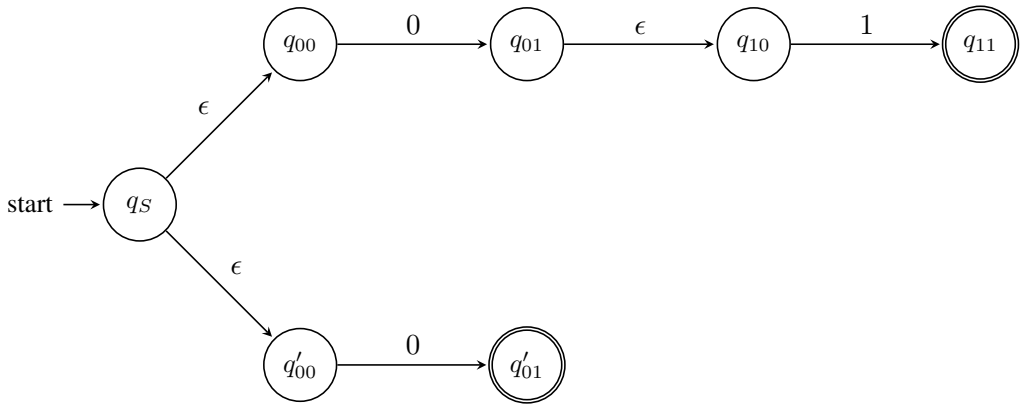
- 1:



- 01:

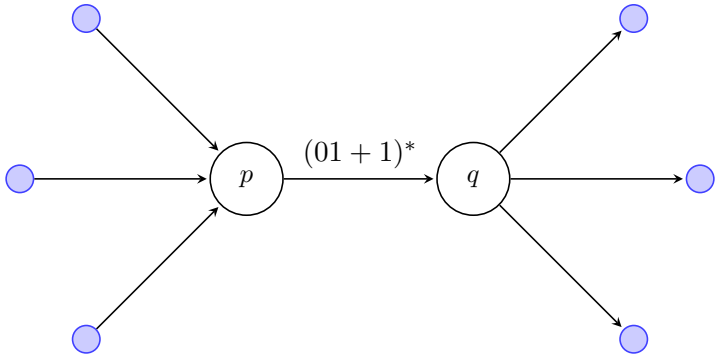


- $01 + 0$:



To prove Lemma 1.3, we first define "Generalized NFA" (GNFA) which takes regular expressions for its transitions:

e.g:

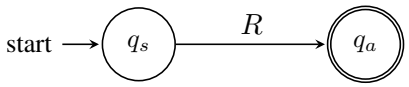


A GNFA also has the following properties:

- The start state does not have any incoming transitions.
- The accepting state (there is only one accepting state) does not have any outgoing transitions.
- Except for these, there is exactly one transition from every state to every other state.

Proof of Lemma 1.3. We will:

1. Convert the DFA into a GNFA (generalized NFA).
2. Reduce the number of states in the GNFS one by one until just 2 states remain.



which is equivalent to regular expression R.

Definition: A GNFA is a 5-tuple $(Q, \Sigma, \delta, q_s, q_a)$:

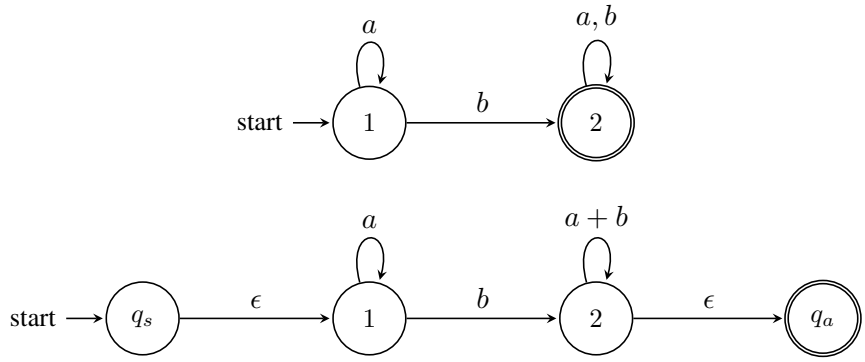
- Q is a finite set of states.
- Σ is the alphabet.
- q_s is the start state.
- q_a is the accepting state.

- $\delta : (Q - \{q_a\}, Q - \{q_s\}) \to R$ is the transition function, where R denotes the set of regular expressions over Σ.

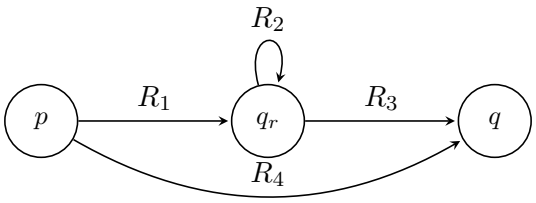
To convert a DFA to GNFA:

- Add two new states q_s and q_a.
- Link q_s to the DFA's start state by ϵ. Link the DFA's accept state to q_a by ϵ.
- Assume an $\emptyset$-transition for the missing arcs (just for the completeness of GNFA). ∎

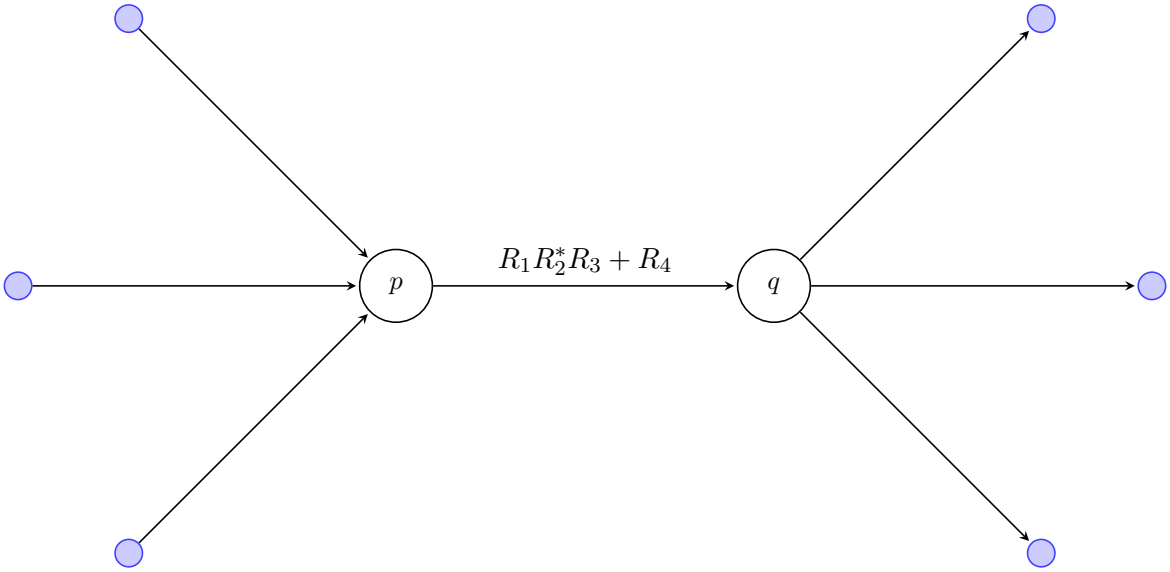
Example Converting DFA.



We just assume $\emptyset$-transitions; no need to write them. To reduce te GNFA by removing a state q_r:



update the transition function as:



and do this for all remaining states.

Reduce (G, q_r):
(Reduce GNFA G by removing state q_r)
Given $G = (Q, \Sigma, \delta, q_s, q_a)$, output $G = (Q', \Sigma, \delta', q_s, q_a)$ where:

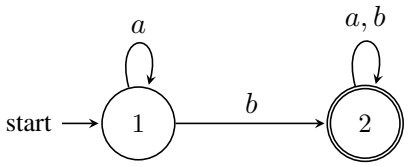
$$Q' = Q - \{q_r\}$$
$$\delta'(p, q) = \delta(p, q_r)\delta(q_r, q_r)^*\delta(q_r, q) + \delta(p, q)$$

Claim: $L(G') = L(G)$

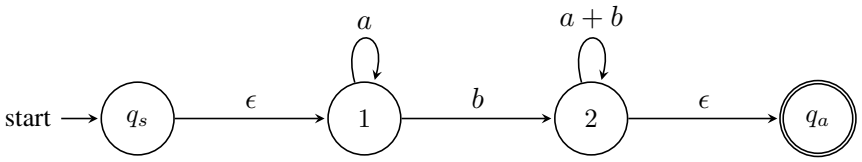
Algorithm 1 Convert(G)

if $|Q| = 2$ **then**
  return $\delta(q_s, q_a)$
else
  choose any state $q \in Q - \{q_s, q_a\}$
  $G' \leftarrow Reduce(G, q)$
  return $Convert(G')$
end if

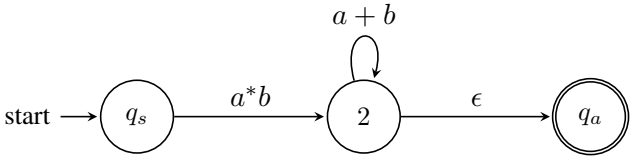
Example Convert DFA into a RE
DFA:



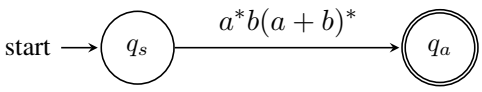
GNFA:



remove 1:



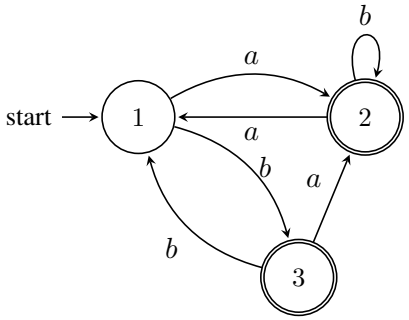
remove 2:



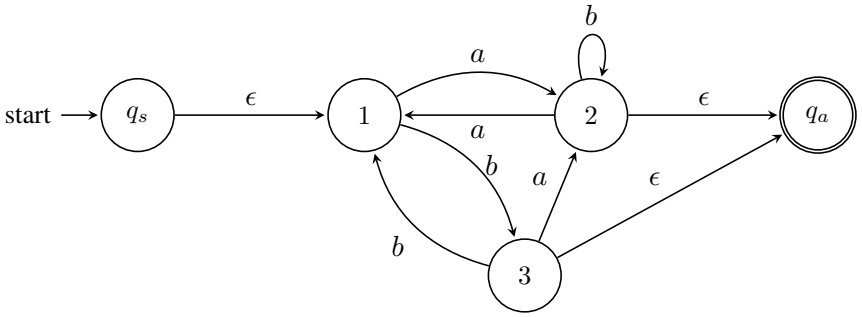
$R = a^*b(a + b)^*$

Example Convert DFA into a RE

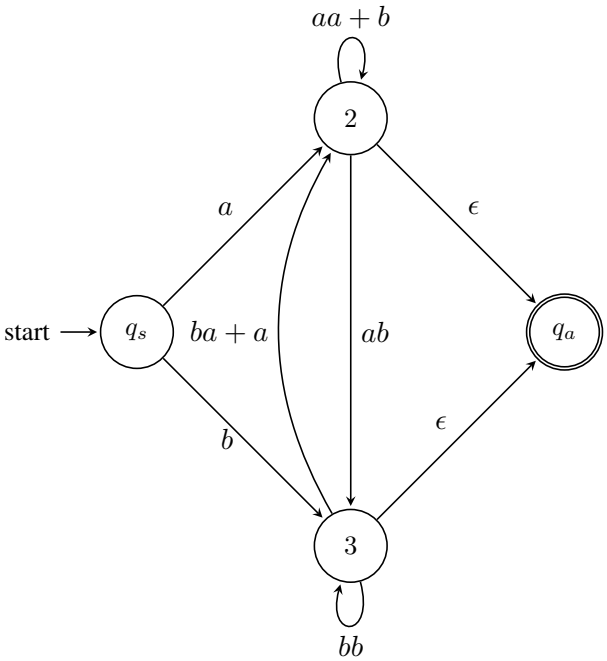
DFA:



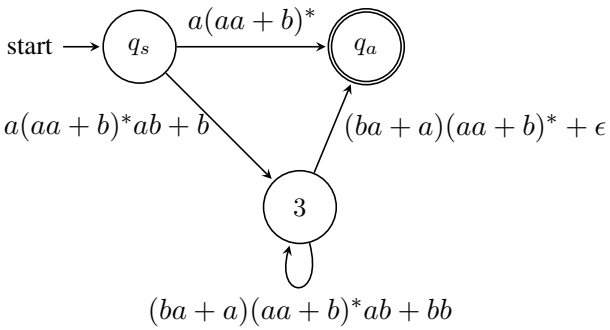
GNFA:



remove 1:



remove 2:



$$R = (a(aa + b)^*ab + b)((ba + a)(aa + b)^*ab + bb)^*((ba + a)(aa + b)^* + \epsilon) + a(aa + b)^*$$

Rule: $\delta'(p,q) = \delta(p,q_r)[\delta(q_r,q_r)]^*\delta(q_r,q) + \delta(p,q)$
Steps:
Remove 1:

$$\begin{aligned}
\delta(q_s,2) &= \delta(q_s,1)[\delta(1,1)]^*\delta(1,2) + \delta(q_s,2)\\
&= \epsilon\ a\\
&= a\\
\delta(q_s,3) &= \delta(q_s,1)[\delta(1,1)]^*\delta(1,3) + \delta(q_s,3)\\
&= \epsilon\ b\\
&= b
\end{aligned}$$

$$\begin{aligned}
\delta(2,2) &= \delta(2,1)[\delta(1,1)]^*\delta(1,2) + \delta(2,2)\\
&= aa + b\\
\delta(2,3) &= \delta(2,1)[\delta(1,1)]^*\delta(1,3) + \delta(2,3)\\
&= ab
\end{aligned}$$

$$\begin{aligned}
\delta(3,2) &= \delta(3,1)[\delta(1,1)]^*\delta(1,2) + \delta(3,2)\\
&= ba + a\\
\delta(3,3) &= \delta(3,1)[\delta(1,1)]^*\delta(1,3) + \delta(3,3)\\
&= bb
\end{aligned}$$

Remove 2:

$$\begin{aligned}
\delta(q_s,3) &= \delta(q_s,2)[\delta(2,2)]^*\delta(2,3) + \delta(q_s,3)\\
&= a(aa+b)^*ab + b\\
\delta(q_s,q_a) &= \delta(q_s,2)[\delta(2,2)]^*\delta(2,q_a) + \delta(q_s,q_a)\\
&= a(aa+b)^*\\
\delta(3,3) &= \delta(3,2)[\delta(2,2)]^*\delta(2,3) + \delta(3,3)\\
&= (ba+a)(aa+b)^*ab + bb\\
\delta(3,q_a) &= \delta(3,2)[\delta(2,2)]^*\delta(2,q_a) + \delta(3,q_a)\\
&= (ba+a)(aa+b)^*
\end{aligned}$$

Remove 3:

$$\begin{aligned}
\delta(q_s,q_a) &= \delta(q_s,3)[\delta(3,3)]^*\delta(3,q_a) + \delta(q_s,q_a)\\
&= (a(aa+b)^*ab+b)((ba+a)(aa+b)^*ab+bb)^*((ba+a)(aa+b)^*) + a(aa+b)^*
\end{aligned}$$

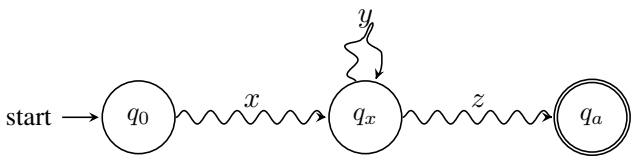
1.2 Pumping Lemma

A simple but powerful idea to prove that a language is not regular (i.e. cannot be accepted by a DFA).

Consider a DFA M with n states. Then, for any string w where $|w| \geq n$ has to contain some cycle in its computation.

$$w = a_1 a_2 \dots a_m, m \geq n$$
$$q_0 \to q_1 \to \dots q_m \quad has\ m+1 > n\ states\ on\ the\ path.$$

Then by pigeonhole principle, this path has some re-visited state:



$xyz \in L$
$xy^2z \in L$
$\dots$
$xy^iz \in L\ for\ all\ i \geq 0$
$Note:\ xy^0z = xz \in L$

Lemma 1.4 *(Pumping Lemma)*

If L is a RL, then there is some integer p ("pumping length") where if $w \in L$ and $|w| \geq p$, then w can be divided into three. $w = xyz$, such that:

1. $xy^iz \in L, \forall i \geq 0$
2. $|y| > 0$
3. $|xy| \leq p$

Proof of Lemma 1.4

If L is regular, it must be accepted by some DFA=$Q, \Sigma, \delta, q_0, F$. Let $p = |Q|$.

- The first two conditions follow from the previous argument.
- The third condition says that such a cycle can be found within the first p transitions. ∎

To prove a language is regular: You are to proof that for **all sufficiently long strings (w)** in language there is **at-least one way (y)** to generate new strings in the language by repeating looping part any number (i) of times.

To prove a language is not regular: You are to find **at least one sufficiently long string (w)** in language such that there **no choice for any 'y'** so that its possible to generate new strings with all possible repetition (i).

Proving using Pumping Lemma:

	$w \in L, \lvert w\rvert \geq p$	y	i
language is regular	$\forall w \in L$	$\exists y$	$\forall i \geq 0$
language is NOT regular	$\exists w \in L$	$\nexists y$	$\exists i \geq 0$

Example Show the following languages are not regular:

- $L_1 = \{0^n 1^n | n \geq 0\}$

  Assume that L_1 is regular and let p be its pumping length. Consider $w = 0^p 1^p$.

  - Since $|w| \geq p$, its DFA has to contain some cycle (i.e. $w = xyz$ such that $xy^i z \in L_1, \forall i \geq 0$).
  - The cycle is within the first p characters because of the rule $|xy| \leq p$ (i.e., $y = 0^k$ for some $k > 0$), then

  $$xy^2z = 0^{p+k}1^p \notin L_1$$

  Therefore L_1 is not regular. ∎

  Alternatively; consider $w = 0^{\frac{p}{2}} 1^{\frac{p}{2}}$.

  - Since $|w| \geq p$, its DFA has to contain some cycle (i.e., $w = xyz$ such that $xy^i z \in L_1, \forall i \geq 0$). $|xy| \leq p$, therefore there are three possibilities for y:
  - If the cycle is within the first $\frac{p}{2}$ characters (i.e., $y = 0^k$ for some $k > 0$), then

  $$xy^2z = 0^{\frac{p}{2}+k}1^{\frac{p}{2}} \notin L_1$$

  - Similarly, $y = 1^k$ is not possible:

  $$xy^2z = 0^{\frac{p}{2}}1^{\frac{p}{2}+k} \notin L_1$$

  - Finally, if y contains both 0 and 1 ($y = 0^k 1^l$; $k, l > 0$, then if we **pump up**, we get:

  $$xy^2z = 0^{\frac{p}{2}-k}(0^k 1^l)^2 1^{\frac{p}{2}-l} = 0^{\frac{p}{2}}1^l 0^k 1^{\frac{p}{2}} \notin L_1$$

  That is, we will have some mixed string of 0s and 1s ($\notin L_1$). ∎

- $L_2 = \{w | n_0(w) = n_1(w)\}$

  Assume that L_2 is regular and let p be its pumping length. Consider $w = 0^p 1^p$.

  - By the 3^{rd} condition, w must contain a cycle within the first p characters.
  - That is, if $y = 0^k$ for some $k > 0$,
  - Then, $xy^2z = 0^{p+k}1^p \notin L_2$

  Therefore L_2 cannot be regular.

- $L_3 = \{w | w = w^R\}$ (i.e., w is a palindrome)

  Assume that L_3 is regular and let p be its pumping length. Consider $w = 0^p 10^p$.

  - We must have y among the first group of 0s; $y = 0^k$, $k > 0$.
  - Then $xy^2z = 0^{p+k}10^p \notin L_3$

  Hence L_3 is not regular.

- $L_4 = \{w | w = vv^R,\ v \in \{0,1\}^*\}$

  Consider $w = 0^p 110^p$. Left as exercise.

- $L_5 = \{w | w = vv,\ v \in \{0,1\}^*\}$

  Consider $w = 0^p 10^p 1$. $y = 0^k$ for some $k > 0$. $xy^2z = 0^{p+k}10^p 1 \notin L_5$.

- $L_6 = \{0^m 1^n | m \geq n\}$.

  Consider $w = 0^p 1^p$. Then $y = 0^k,\ k > 0$.

  "Pump down": $xy^0z = xz = 0^{p-k}1^p \notin L_6$.

  Also consider $w = 0^p 1^{p-1}$. Then $y = 0^k,\ k > 0$.

  "Pump down": $xz = 0^{p-k}1^{p-1}$. This is in L_6 for some k, and not in L_6 for other k.

- $L_7 = \{0^m 1^n | m \geq n\}$.

  Alternatively, we can use the closure property: if L^R is not regular, then L is not regular.

  $L_7^R = \{1^n 0^m | n \leq m\}$.

  Consider $w = 1^p 0^p$. Then $y = 1^k,\ k > 0$.

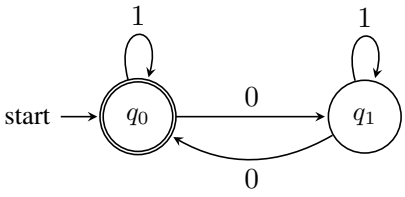
  Pump up: $xy^2z = 1^{p+k}0^p \notin L_7^R$.

- $L_8 = \{w | n_0(w)\ is\ even\}$.

  Consider $w = 0^{2p}$. Then $y = 0^k,\ k > 0$.

  Pump up $xy^2z = 0^{2p+k}$. If k is odd, $xy^2z \notin L_8$. Hence, L_8 is not regular.

  $\Rightarrow$ **WRONG!**



- $L_9 = \{0^m 1^n | m \neq n\}$.

  Consider $L_9' = \{0^m 1^n | m = n\}$.

  $L_9' = 0^*1^* - L_9$. We know that L_9' is not regular. If L_9 were regular, then by closure under "-", L_9' would be regular. Since L_9' is not regular, L_9 is not regular.

- $L_{10} = \{w | n_0(w) \neq n_1(w)\}$.

  Consider $\overline{L_{10}}$...

- $L_{11} = \{0^{n^2}, n \geq 1\} = \{0, 0000, 000000000, \ldots\}$.

  Consider $w = 0^{p^2}$. Then $y = 0^k,\ k > 0$.

  Pump up: $xy^2z = 0^{p^2+k}$. Note that $p^2 + k$ cannot be a square. The next square is $(p+1)^2 = p^2 + 2p + 1 > p^2 + k$ given that $k < p$.

- $L_{12} = \{0^n | n\ is\ a\ prime\ number\}$.

  Assume L_{12} is regular and let p be its pumping length.

  Consider $w = 0^q$ for some prime $q \geq p$. Then $y = 0^k,\ k > 0$.

  $$\begin{aligned} xyyz &= 0^{q+k} \\ xyyyz &= 0^{q+2k} \\ &\cdot \\ &\cdot \\ xy^iz &= 0^{q+(i-1)k} \end{aligned}$$

  Then for $i = q + 1$, we have $xy^{q+1}z = 0^{q+qk} = 0^{q(1+k)} \notin L_{12}$ for any $k > 0$ since $q(1+k)$ cannot be a prime number.

- $L_{13} = \{0^m 1^n | m \neq n\}$.
  Consider $w = 0^p 1^{p+p!}$. Then $y = 0^k,\ 1 \leq k \leq p$.
  Then, $xy^i z = 0^{p+(i-1)k} 1^{p+p!}$, and for $i - 1 = \frac{p!}{k}$ (i.e. $i = \frac{p!}{k} + 1$).
  We have $xy^i z = 0^{p+p!} 1^{p+p!} \notin L_{13}$.

- $L_{14} = \{w | w^R \neq w\}$.
  Consider $w = 0^p 10^{p+p!}$. Then $y = 0^k,\ k > 0$.
  $xy^i z = 0^{p+(i-1)k} 10^{p+p!}$, and for $i = \frac{p!}{k} + 1$, we have:
  $0^{p+p!} 10^{p+p!} \notin L_{14}$.

- $L_{15} = \{0^m 1^n |\ gcd(m, n) = 1\}$. (i.e. m and n are relatively prime)
  For some prime $q \geq p + 2$.
  $w = 0^{q+(q-1)!} 1^{(q-1)!}$. Then $y = 0^k,\ 1 \leq k \leq p$.
  Pump down: $0^{q-k+(q-1)!} 1^{(q-1)!} \notin L_{15}$. Note that $(q - k)$ is a non-trivial divisor for both sides.

1.3 Decision Questions about RLs

Given an n-state DFA, how can you tell whether its language is:

1.3.1 Finite/Infinite

Check all strings with length $n \leq |w| \leq 2n$. If any such $w \in L$, then L is infinite. Else L is finite.

Theorem 1.5 *For an n-state DFA M, $L(M)$ is infinite iff $L(M)$ contains some string of length ℓ where $n \leq \ell \leq 2n$.*

Proof First, if $L(M)$ contains a string w of length $[n, 2n)$, by the pumping lemma, we can produce infinitely many strings in $L(M)$: $xy^i z, \forall i \geq 0$ for some partition $w = xyz$.

Conversely, if $L(M)$ is infinite, then $\exists w', |w'| \geq n$. If $|w'| < 2n$ we are done. If $|w'| \geq 2n$, then from this w' we can obtain some w, $n \leq |w| < 2n$ by removing the cycles; since each basic cycle will have length $\leq n$. ∎

1.3.2 Empty

Check all strings with length $|w| < n$. $L(M)$ is empty iff no such w is in $L(M)$.